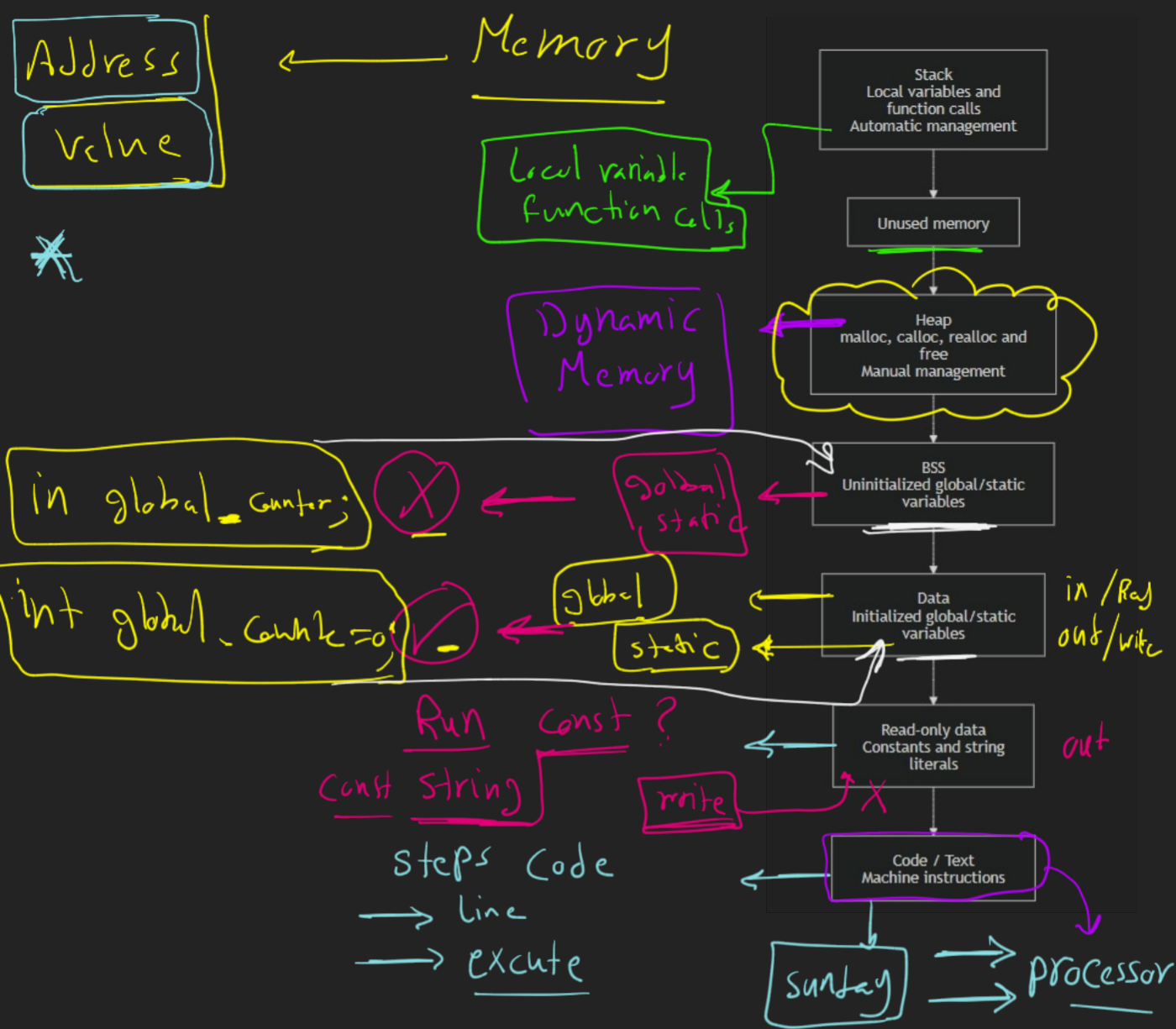




* Heap Dynamic Memory

Memory can be allocated during
the Run time.

* Static Memory

Memory could be allocated during
the Compile time.

* Functions

include < std lib.h >

malloc()



calloc()



realloc()



free()



→ malloc()

function out

→ return →
→ NULL

function input

→ size in Bytes

char [Vg]



Address

*ptr = (Value + 1)

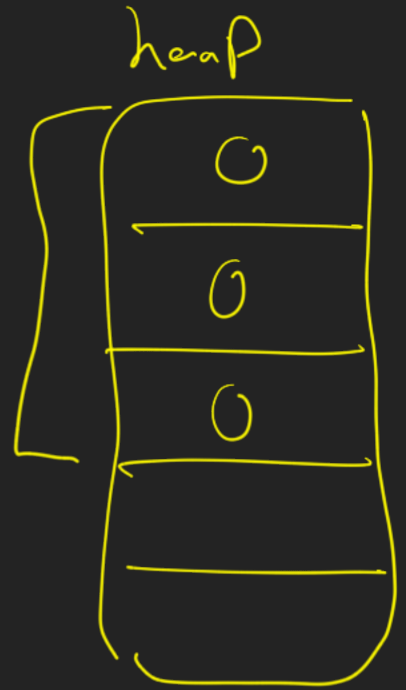
malloc (n * size (datatype))

char *ptr = malloc (n * sizeof(char))

calloc (,)



↳ Initialization = 0



which is better ?

malloc

calloc

Fast



Memory

Tract



realloc

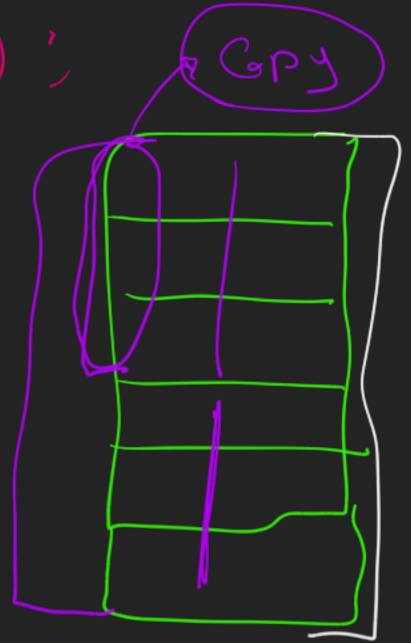
3

6

int x = 3;

ptr malloc (n * sizeof(int);

realloc (ptr,);



2K Bytes

Computer

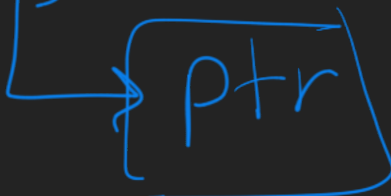
return

void *ptr

NULL; $\longrightarrow$ No Memory

مفید سا رہا ہے

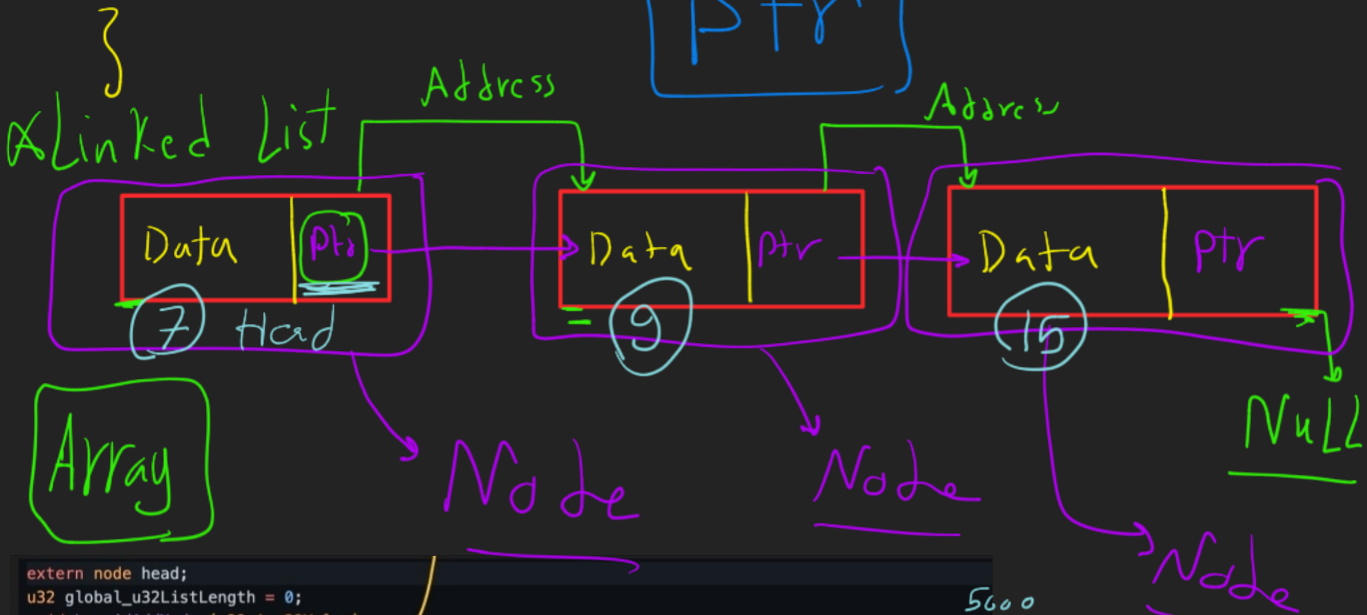
Free()



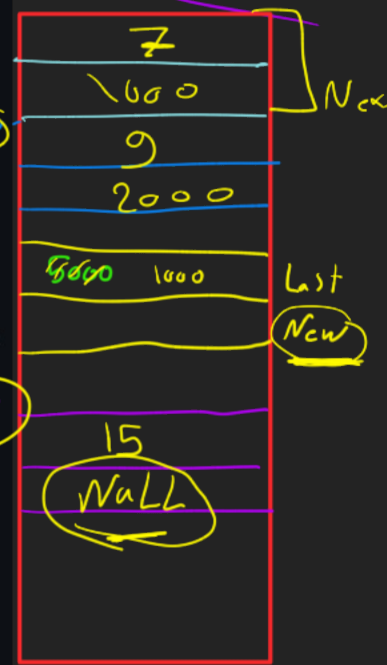
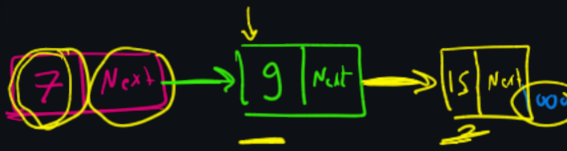
```
typedef struct {
    int *ptr;
}
```

Nested struct

ptr



```
extern node head;
u32 global_u32ListLength = 0;
void L_voidAddNode (u32 A_u32Value)
{
    if (global_u32ListLength)
    {
        head.value = A_u32Value;
        head.next = NULL;
    }
    else
    {
        node *last;
        // Allocate New Node
        node *new = (node *) malloc (sizeof(node));
        // Assign the received value to the new Node
        new->value = A_u32Value;
        // Make the New Node Next pointer points to NULL, as it will be added at the end of the list
        new->next = NULL;
        last = &head;
        // Search for last node
        while ((last->next) != NULL)
        {
            last = (last->next);
        }
        last->next = new;
    }
    // Increase the list length by one node
    global_u32ListLength++;
    return;
}
```



```
typedef struct NODE_t node;
struct NODE_t
{
    u32 value;
    node *next;
};
```

Function U32 - void

variable int

U32_MoveCursor

U8 G L | U8 | 3

Move_Cursor

Void Init

L_U8_character